Manual técnico de LoncheS



Fecha de publicación

10/08/2024

Contenido

Introducción ____ ***4***

Alcance ____ ***4***

Objetivo general: ____ **4**

Objetivos específicos: ____ **4**

Límites de la documentación: ____ **5**

Descripción de procesos ____ ***5***

Diagramas UML ____ ***6***

Diagramas de casos de uso ____ **6**

Autenticación: ____ 6

Gestión ingredientes: ____ 7

Gestión recetas: ____ 7

Administración de usuarios ____ ***8***

- Modificación de Información ____ 8
- Eliminación de Usuarios ____ 8
- Definición de Roles ____ 8

Modelo relacional de la base de datos ____ ***8***

Diccionario de datos ____ **10**

Descripción de la plataforma ____ ***10***

- **Lenguaje de Programación** ____ **10**
- **Interfaz de Usuario** ____ **10**
- **Librerías y Herramientas** ____ **10**

Documentación del código fuente ____ ***11***

Repositorio ____ **11**

app.Http.Controllers ____ **12**

app.Http.Controllers.Auth ____ 12

app.Http.Middleware ____ **12**

app.Models ____ **13**

databases.migrations ____ **13**

databases.seeders ____ **14**

public.assets.css ____ **14**

public.js ____ **15**

resources.views ____ **15**

routes ____ **15**

Estructura de directorios ____ **16**

Relación de programas ____ **17**

Descripción de los acuerdos de niveles de servicios o ANS ____ ***19***

Nivel de Servicio ____ **19**

Disponibilidad ____ **19**

Soporte ____ **19**

Responsabilidades ____ **19**

Introducción

El presente manual técnico tiene como objetivo proporcionar una guía detallada sobre la estructura, configuración y funcionamiento de la aplicación LoncheS. Esta plataforma ha sido diseñada para facilitar a los usuarios la gestión de recetas saludables y la organización de ingredientes disponibles en su inventario, con el propósito de planificar loncheras nutritivas de manera eficiente.

A lo largo de este documento, se describirán los componentes técnicos del sistema, así como los pasos necesarios para su instalación, configuración y mantenimiento. El manual está dirigido a desarrolladores y administradores del sistema, y ofrece una visión general de las tecnologías empleadas, la arquitectura del proyecto, los módulos principales y la interacción con la base de datos

LoncheS busca ofrecer una experiencia fluida y segura para sus usuarios, permitiendo la autenticación y acceso a funcionalidades específicas según su rol dentro del sistema. Este documento servirá como referencia técnica para asegurar que el sistema funcione de manera óptima y que se puedan realizar futuras actualizaciones y mejoras de manera eficiente.

Alcance

Objetivo general:

El sistema de información LoncheS está diseñado para ayudar a los usuarios a alimentarse de manera saludable mediante la gestión eficiente de recetas y el inventario de ingredientes. Además, garantiza una experiencia de usuario óptima al permitir la búsqueda de recetas a partir de la selección de ingredientes disponibles. El sistema es adaptable a dispositivos móviles y ofrece una interfaz intuitiva, facilitando la navegación por las diferentes funcionalidades de la plataforma. En caso de dudas o inconvenientes, los usuarios pueden ponerse en contacto con el equipo de soporte a través de correo electrónico.

Objetivos específicos:

- Facilitar la gestión de recetas saludables, permitiendo a los usuarios buscar y seleccionar recetas según sus necesidades.
- Proporcionar una herramienta para gestionar el inventario de ingredientes, ayudando a los usuarios a controlar los alimentos disponibles en su alacena.
- Garantizar una experiencia de navegación sencilla y adaptable en todos los dispositivos, especialmente en móviles.
- Facilitar la administración del sistema por parte de los administradores, con acceso a la gestión de recetas, ingredientes y usuarios.

- Implementar una gestión de roles que garantice que cada usuario tenga acceso solo a las funcionalidades que corresponden a su perfil.

Límites de la documentación:

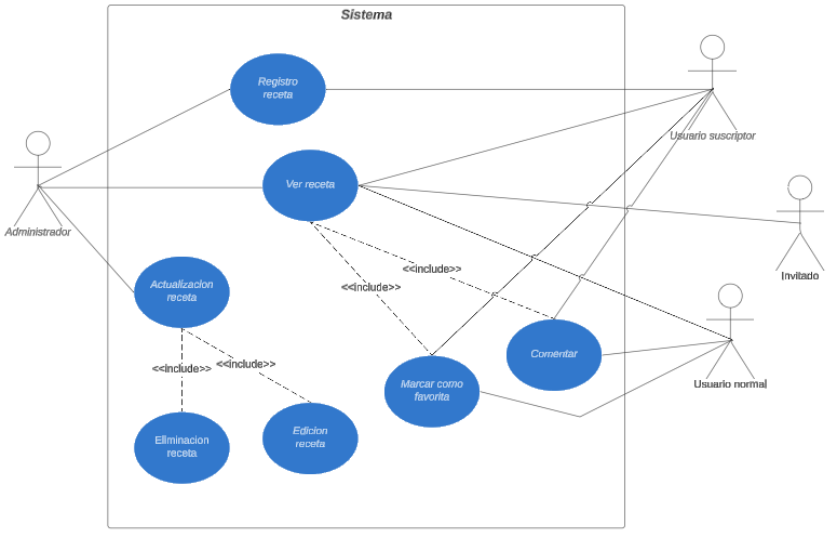
Este documento se centra en la configuración y operación técnica de LoncheS. No aborda temas relacionados con la experiencia del usuario final, estrategias nutricionales específicas o procesos administrativos externos a la aplicación, como la gestión de pagos o acuerdos comerciales.

Descripción de procesos

El sistema de información LoncheS está diseñado para optimizar y gestionar los procesos clave relacionados con la búsqueda de recetas saludables, la administración de recetas y el control de inventarios. Esta sección detalla los procesos específicos que la plataforma respalda, ofreciendo una descripción clara de cómo LoncheS facilita las operaciones diarias de los usuarios, mejorando la eficiencia en la organización de alimentos y la experiencia de uso.

Los principales procesos son:

- **Gestión de Usuarios:** El sistema permite la autenticación segura de los usuarios, la gestión de roles y permisos, y el acceso a funcionalidades específicas según el tipo de usuario (administrador o suscriptor).



- **Control de Inventario de Ingredientes:** Los usuarios pueden gestionar los ingredientes disponibles en su alacena y nevera, registrando las cantidades y unidades correspondientes para planificar sus recetas de manera eficiente.

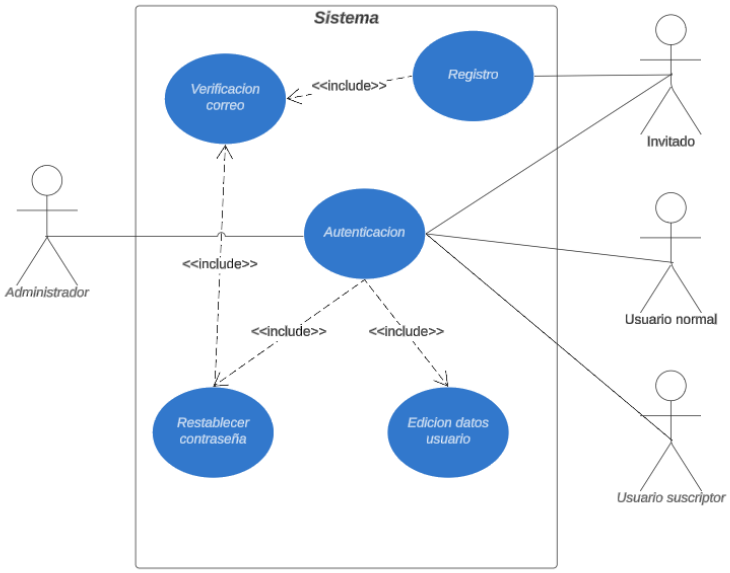
- **Gestión de Recetas:** El sistema permite agregar, editar y eliminar recetas, así como marcar las favoritas y agregar comentarios sobre las recetas.
- **Búsqueda de Recetas:** La plataforma ofrece una búsqueda dinámica de recetas, permitiendo a los usuarios filtrar por ingredientes, categorías y otros criterios relevantes.
- **Administración del Sistema:** Los administradores pueden gestionar los ingredientes del sistema, usuarios registrados, y realizar un seguimiento de las métricas clave de la plataforma.

Cada uno de estos procesos está diseñado para garantizar una experiencia fluida y eficiente, facilitando a los usuarios el control de sus ingredientes de manera práctica y organizada.

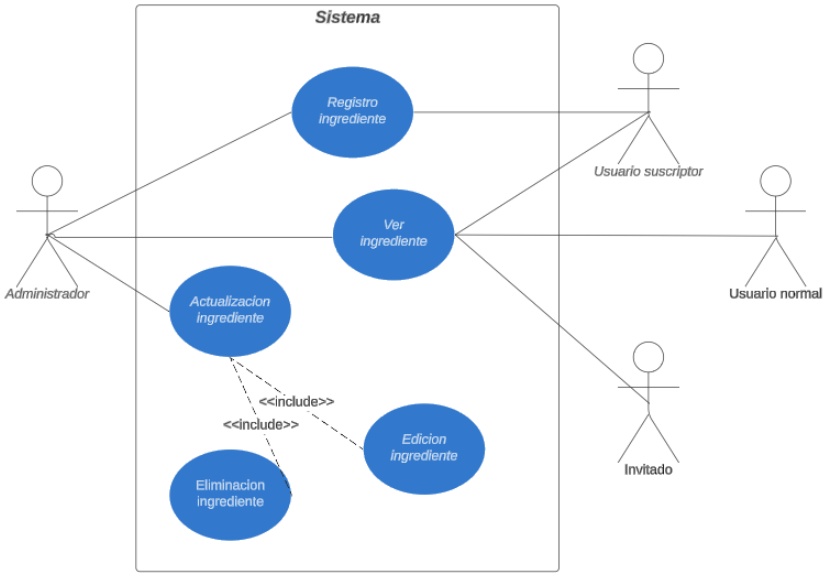
Diagramas UML

Diagramas de casos de uso

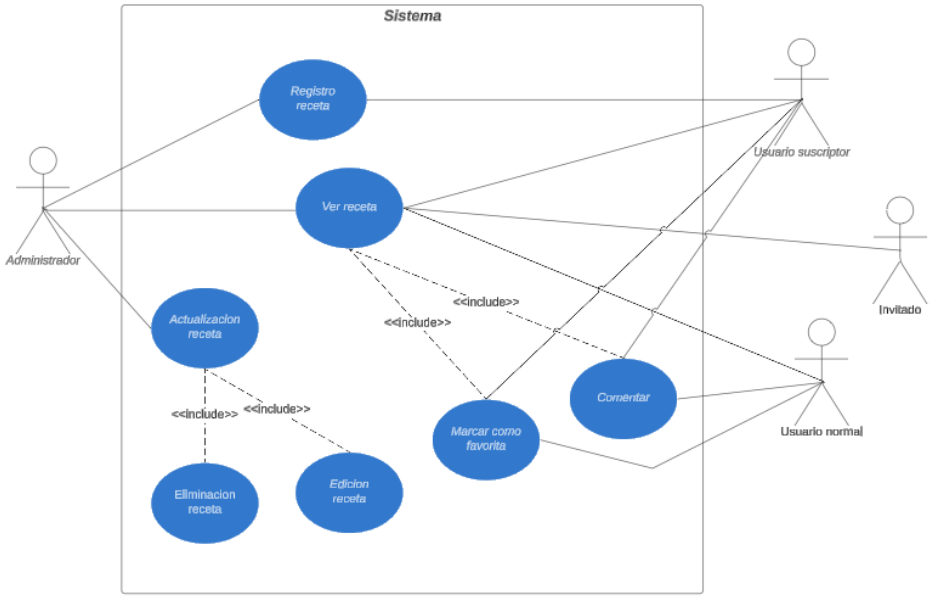
Autenticación:



Gestión ingredientes:



Gestión recetas:



Administración de usuarios

En LoncheS, el módulo de administración de usuarios es esencial para asegurar un control efectivo sobre los accesos y permisos dentro del sistema. Este módulo permite a los administradores gestionar de manera integral la información y los privilegios de los usuarios, que pueden incluir roles como Normal, Suscriptor y Administrador. Las funcionalidades clave del módulo son las siguientes:

A. Modificación y Eliminación de Usuarios.

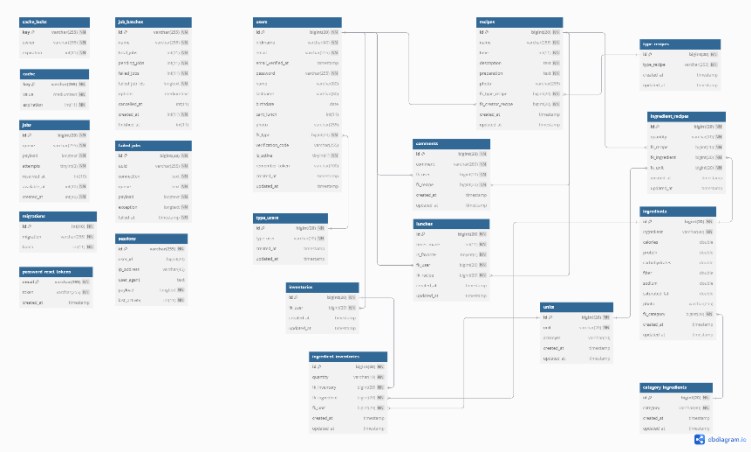
- **Modificación de Información:** Los administradores tienen la capacidad de actualizar la información de los usuarios existentes, como modificar nombres de usuario, correos electrónicos y otros detalles relevantes del perfil.
- **Eliminación de Usuarios:** En caso de que un usuario ya no requiera acceso al sistema, el administrador puede desactivar su cuenta o eliminarla, garantizando así la gestión adecuada de accesos.

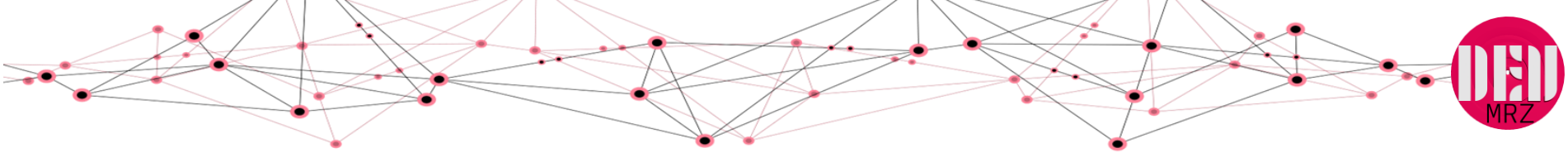
B. Gestión de Roles de Usuario.

- **Definición de Roles:** Este apartado permite gestionar los roles dentro de la plataforma, como Usuario y Administrador. Cada rol tiene un conjunto específico de permisos y funcionalidades asignadas que determinan el acceso a las distintas áreas del sistema.

Este módulo garantiza que LoncheS mantenga un control riguroso sobre quién puede acceder y modificar diferentes aspectos del sistema, asegurando así la integridad y seguridad de la plataforma.

Modelo relacional de la base de datos





cache_locks

key	varchar(255) NN
owner	varchar(255) NN
expiration	int(11) NN

cache

key	varchar(255) NN
value	mediumtext NN
expiration	int(11) NN

jobs

id	bigint(20) NN
queue	varchar(255) NN
payload	longtext NN
attempts	tinyint(3) NN
reserved_at	int(10)
available_at	int(10) NN
created_at	int(10) NN

migrations

id	int(10) NN
migration	varchar(255) NN
batch	int(11) NN

password_reset_tokens

email	varchar(255) NN
token	varchar(255) NN
created_at	timestamp

job_batches

id	varchar(255) NN
name	varchar(255) NN
total_jobs	int(11) NN
pending_jobs	int(11) NN
failed_jobs	int(11) NN
failed_job_ids	longtext NN
options	mediumtext
cancelled_at	int(11)
created_at	int(11) NN
finished_at	int(11)

failed_jobs

id	bigint(20) NN
uuid	varchar(255) NN
connection	text NN
queue	text NN
payload	longtext NN
exception	longtext NN
failed_at	timestamp NN

sessions

id	varchar(255) NN
user_id	bigint(20)
ip_address	varchar(45)
user_agent	text
payload	longtext NN
last_activity	int(11) NN

users

id	bigint(20) NN
nickname	varchar(40) NN
email	varchar(255) NN
email_verified_at	timestamp
password	varchar(255) NN
name	varchar(80)
lastname	varchar(80)
birthdate	date
cant_lunch	int(11)
photo	varchar(255)
fk_type	bigint(20) NN
verification_code	varchar(255)
is_active	tinyint(1) NN
remember_token	varchar(100)
created_at	timestamp
updated_at	timestamp

type_users

id	bigint(20) NN
type_user	varchar(20) NN
created_at	timestamp
updated_at	timestamp

inventories

id	bigint(20) NN
fk_user	bigint(20) NN
created_at	timestamp
updated_at	timestamp

ingredient_inventories

id	bigint(20) NN
quantity	varchar(10) NN
fk_inventory	bigint(20) NN
fk_ingredient	bigint(20) NN
fk_unit	bigint(20) NN
created_at	timestamp
updated_at	timestamp

recipes

id	bigint(20) NN
name	varchar(255) NN
time	int(11) NN
description	text NN
preparation	text NN
photo	varchar(255)
fk_type_recipe	bigint(20) NN
fk_creator_recipe	bigint(20) NN
created_at	timestamp
updated_at	timestamp

comments

id	bigint(20) NN
comment	varchar(280) NN
fk_user	bigint(20) NN
fk_recipe	bigint(20) NN
created_at	timestamp
updated_at	timestamp

lunches

id	bigint(20) NN
times_made	int(11) NN
is_favorite	tinyint(1) NN
fk_user	bigint(20) NN
fk_recipe	bigint(20) NN
created_at	timestamp
updated_at	timestamp

units

id	bigint(20) NN
unit	varchar(20) NN
acronym	varchar(10)
created_at	timestamp
updated_at	timestamp

type_recipes

id	bigint(20) NN
type_recipe	varchar(255) NN
created_at	timestamp
updated_at	timestamp

ingredient_recipes

id	bigint(20) NN
quantity	varchar(10) NN
fk_recipe	bigint(20) NN
fk_ingredient	bigint(20) NN
fk_unit	bigint(20) NN
created_at	timestamp
updated_at	timestamp

ingredients

id	bigint(20) NN
ingredient	varchar(80) NN
calories	double
protein	double
carbohydrates	double
fiber	double
sodium	double
saturated_fat	double
photo	varchar(255)
fk_category	bigint(20) NN
created_at	timestamp
updated_at	timestamp

category_ingredients

id	bigint(20) NN
category	varchar(80) NN
created_at	timestamp
updated_at	timestamp

dbdiagram.io

Diccionario de datos

Para consultar el diccionario de datos de la base de datos relacional del proyecto LoncheS, haga clic en el enlace que se encuentra después de este párrafo. Allí encontrará una descripción detallada de las tablas, columnas y relaciones que estructuran la base de datos, lo que le ayudará a comprender mejor la organización y el manejo de los datos dentro del sistema.

Enlace al diccionario de datos

Descripción de la plataforma

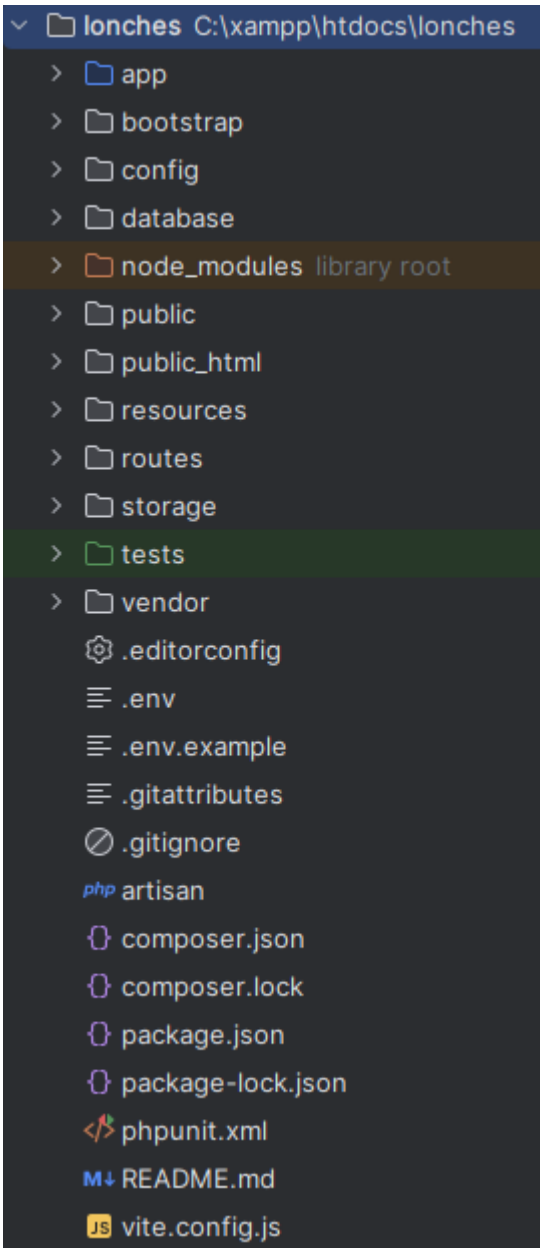
LoncheS se desarrolló utilizando una variedad de tecnologías que nos permiten ofrecer un sistema robusto y eficaz. A continuación, se presentan los principales componentes técnicos utilizados en el desarrollo del proyecto:

- ### Lenguaje de Programación:
  LoncheS está desarrollado en Laravel un framework de PHP, un framework popular para la creación de aplicaciones web del lado del servidor, permitiendo manejar una arquitectura Modelo, Vista, Controlador; lo que en un futuro permitirá escalar la aplicación con mayor facilidad.
- ### Interfaz de Usuario:
  La experiencia de usuario en LoncheS se optimiza mediante el uso de HTML, CSS y Bootstrap. HTML y CSS se utilizan para estructurar y estilizar la interfaz, mientras que Bootstrap asegura que el diseño sea responsivo y se adapte a diversos dispositivos, proporcionando una experiencia de usuario consistente y accesible.
- ### Librerías y Herramientas:
  La funcionalidad de LoncheS se complementa con librerías como jQuery, que facilita la manipulación de elementos en la interfaz de manera ágil y sencilla. Chart.js que permite la implementación de graficas para poder dar informes de manera visual. html2canvas que permite realizar capturas de pantalla de la aplicación desde el frontend. jsPDF que permite generar PDF dentro de la aplicación y posteriormente descargarlos.

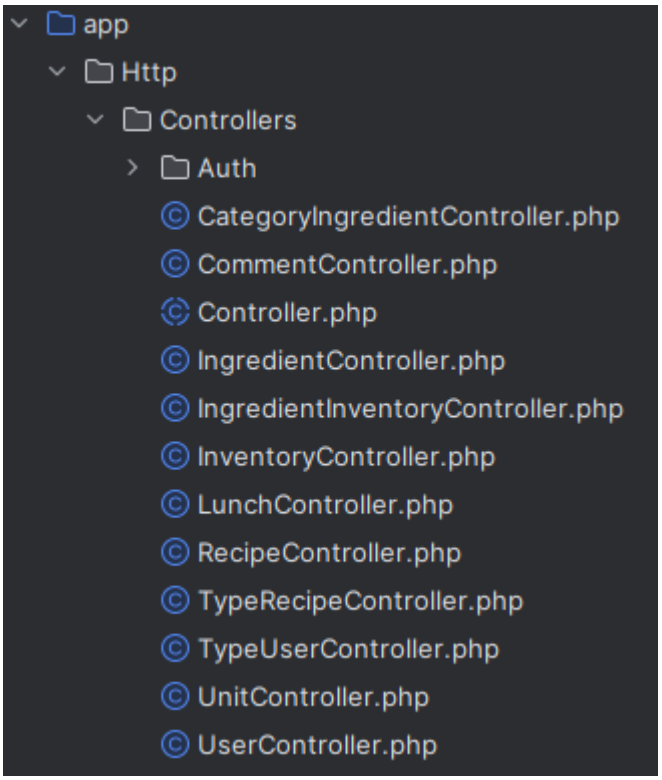
Estas tecnologías y herramientas han sido seleccionadas para garantizar que LoncheS no solo cumpla con los estándares actuales de desarrollo web, sino que también ofrezca un rendimiento eficiente y una interfaz amigable para el usuario.

Documentación del código fuente

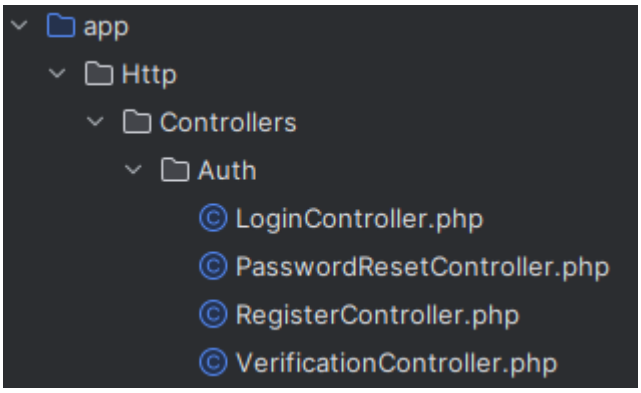
Repositorio



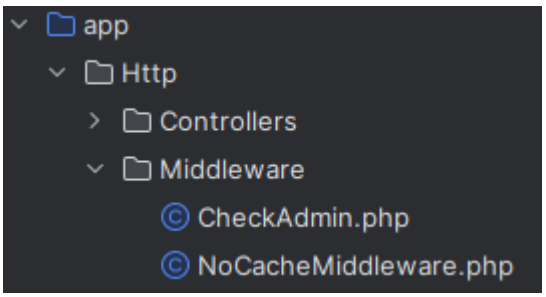
app.Http.Controllers



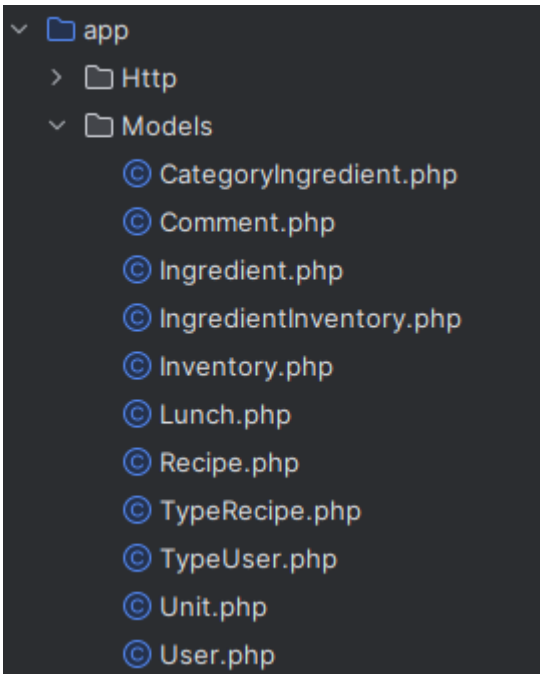
app.Http.Controllers.Auth



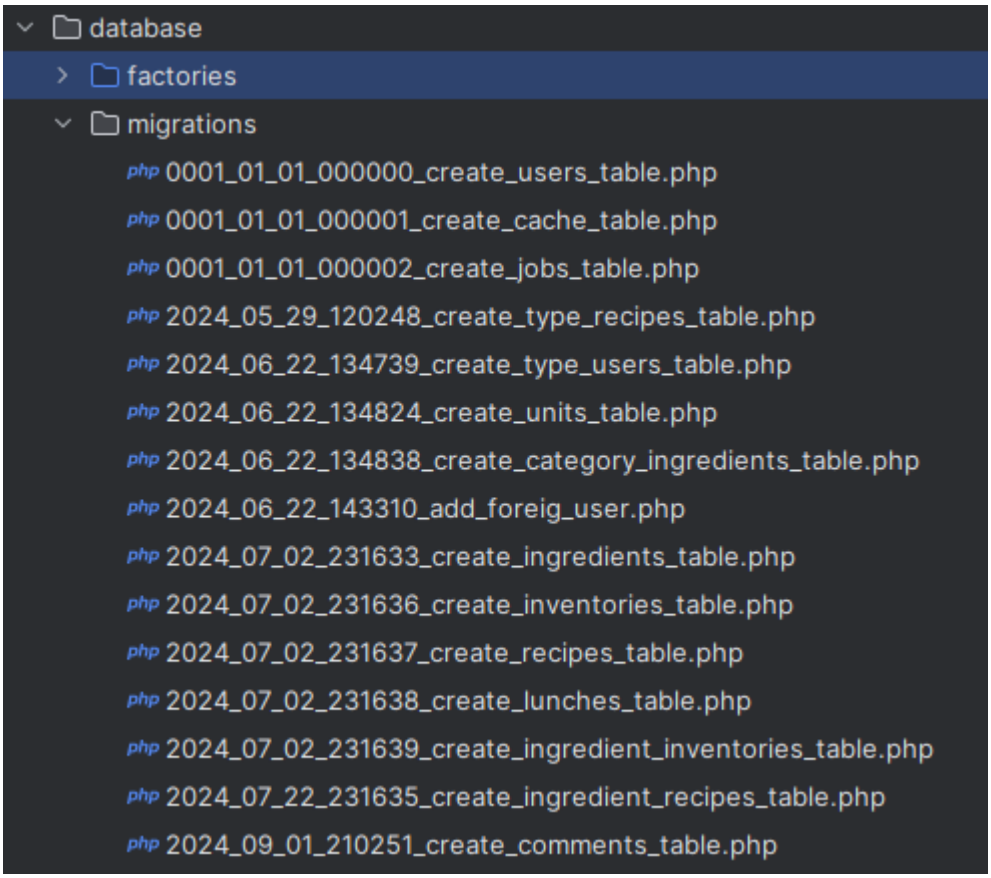
app.Http.Middleware



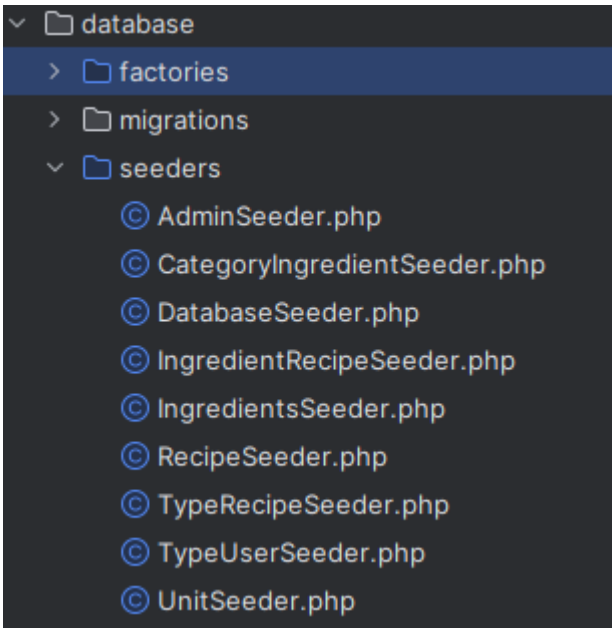
app.Models



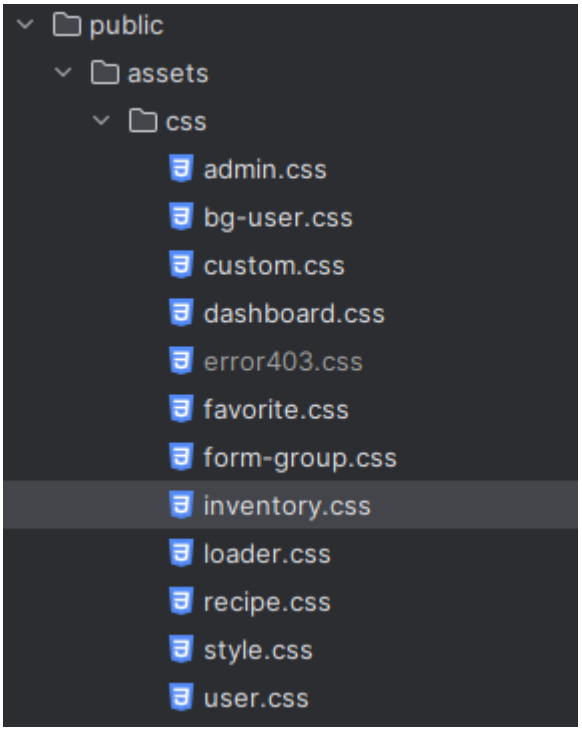
databases.migrations



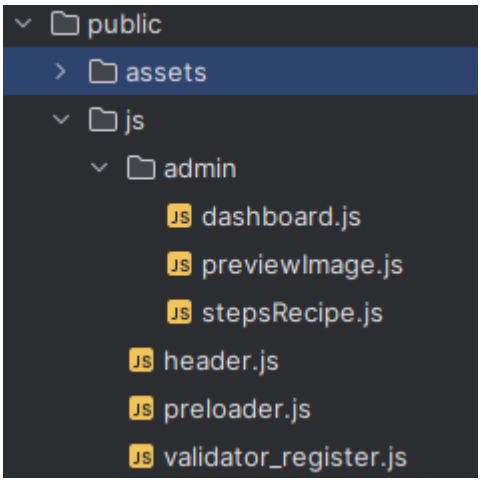
databases.seeders



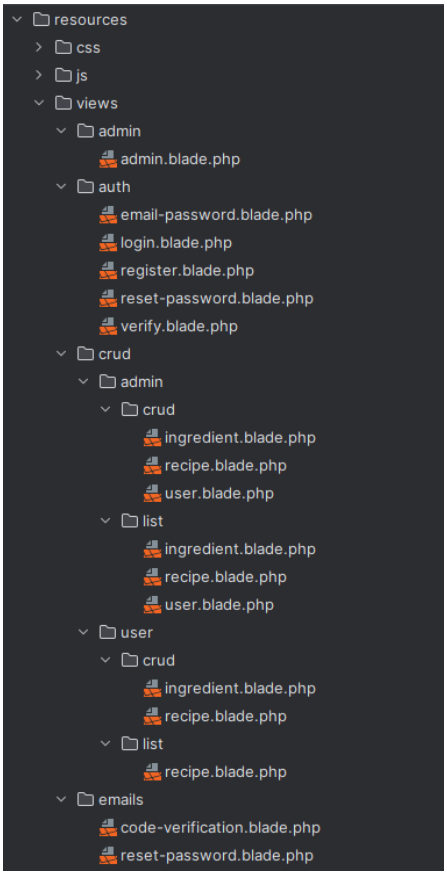
public.assets.css

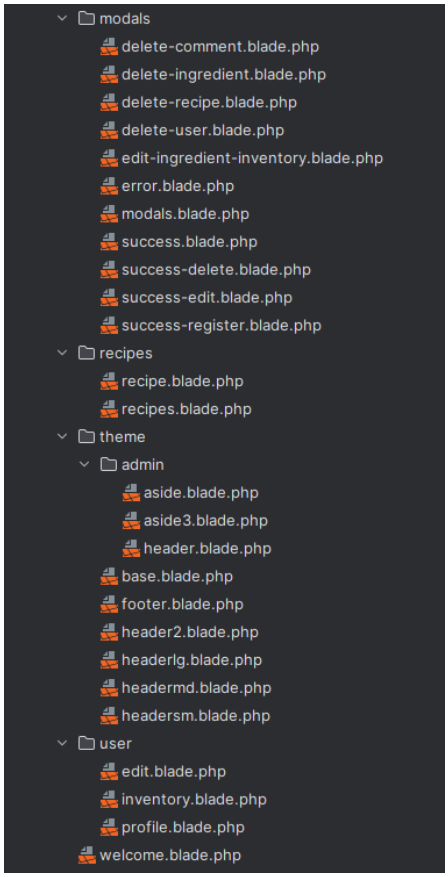


public.js

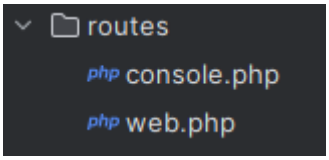


resources.views





routes



Estructura de directorios

Directorio	Descripción
lonches	Directorio donde se encuentran todos los archivos que hacen parte del proyecto
./app	Contiene la lógica principal de la aplicación. Aquí se encuentran los controladores, modelos, middleware, y proveedores de servicios.
./app/Http/Controllers	Almacena los controladores que manejan las solicitudes HTTP.
./app/Http/Middleware	Define el middleware de la aplicación para gestionar solicitudes HTTP.
./app/Models	Contiene los modelos de Eloquent, que interactúan con la base de datos.
./bootstrap	Incluye los archivos de inicialización del framework, como la carga de archivos de configuración y cache.
./config	Contiene todos los archivos de configuración de la aplicación, como información de la base de datos, servicios externos, etc.
./database	Contiene las migraciones, fábricas y seeders para manejar la estructura de la base de datos y datos de prueba.
./database/migrations	Almacena las migraciones de la base de datos.
./database/seeders	Se utiliza para poblar la base de datos con datos iniciales.
./public	El directorio raíz accesible al público donde se encuentran los archivos estáticos EJ: (CSS, JS) y el punto de entrada (index.php).
./public/assets	El directorio accesible al público donde se encuentran los archivos estáticos (CSS, Documentos, Imágenes y recurso graficos).
./public/js	El directorio accesible al público donde se encuentran los archivos estáticos (JS).
./public/uploads	El directorio accesible al público donde se encuentran los archivos estáticos cargados por los usuarios al servidor.
./resources	Almacena los recursos como vistas Blade, archivos de traducción, y activos (CSS, JS) que serán compilados por Webpack o Mix.
./routes	Define todas las rutas de la aplicación en archivos separados como web.php (para rutas web) y api.php (para rutas de API).
./store	Almacena archivos generados por la aplicación, como logs, vistas en caché, y archivos cargados por el usuario.
./tests	Incluye pruebas automatizadas, con carpetas para pruebas unitarias y de características.
./vendor	Contiene las dependencias del proyecto instaladas a través de Composer.
.env	Archivo de configuración del entorno, donde se definen variables sensibles como claves API y configuraciones de base de datos.

Relación de programas

Programa	Lenguaje	Descripción
./app/Http/Controllers/UserController.php	PHP	Este archivo maneja las solicitudes http realizadas a las rutas User por medio de métodos, como index, update, etc.
./app/Http/Controllers/Auth/LoginController.php	PHP	Este archivo maneja las solicitudes http realizadas a las rutas Login por medio de métodos, como showLoginForm, login, etc.
./app/Http/Controllers/IngredientController.php	PHP	Este archivo maneja las solicitudes http realizadas a las rutas Ingredient por medio de métodos, como index, create, store, update, destroy, etc.
./app/Http/Middleware/CheckAdmin.php	PHP	Este archivo maneja las solicitudes http realizadas a las rutas Ingredient por medio de métodos, como handle que valida si el usuario autenticado es o no administrador
./app/Models/Recipe.php	PHP	Este archivo gestiona las interacciones que habrá con la tabla recipes de la base de datos
./app/Models/TypeUser.php	PHP	Este archivo gestiona las interacciones que habrá con la tabla type_users de la base de datos
./database/seeders/UnitSeeder.php	PHP	Este archivo pobla con datos la tabla units de la base de datos
./database/seeders/AdminSeeder.php	PHP	Este archivo realiza un registro en la tabla users de la base de datos, con los datos del administrador
./public/assets/css/admin.css	CSS	Este archivo contiene los estilos que se aplicaran en la vista que incluya dicho archivo y a las etiquetas HTML que sea necesarios

./public/assets/css/loader.css	CSS	Este archivo contiene los estilos que se aplican para lograr el efecto preloader de la aplicación.
./public/js/admin/previewImage.js	JS	Este archivo maneja una función con lo cual se logra previsualizar la imagen seleccionada por el usuario
./public/js/header.js	JS	Este archivo permite modificar el DOM para lograr que dependiendo del tamaño de la pantalla se visualice uno u otro header en la pantalla ayudando esto en el responsive de la aplicacion
./resources/views/welcome.blade.php	HTML, JS y PHP	Este archivo contiene la vista principal de la aplicación.
./resources/views/theme/base.blade.php	HTML	Este archivo contiene la estructura principal de los documentos HTML que después será llamada en diferentes vistas implementando así recursividad.
./resources/views/auth/register.blade.php	HTML	Este archivo contiene la estructura HTML de la vista de registro, donde el usuario podrá registrarse en el sistema.
./routes/web.php	PHP	Este archivo contiene todas las rutas de la aplicación, por medio estas es que se puede acceder a los métodos de los controladores

Descripción de los acuerdos de niveles de servicios o ANS

Nivel de Servicio:

El tiempo máximo de respuesta para las solicitudes de soporte es de 72 horas después de realizada la solicitud en días hábiles. Nos comprometemos a atender las consultas de los usuarios dentro de ese plazo, asegurando una resolución rápida y eficaz de cualquier inconveniente.

Disponibilidad:

La plataforma LoncheS garantiza una disponibilidad del 80% al mes, lo que significa que los usuarios podrán acceder a los servicios de la aplicación la mayor parte del tiempo, con un margen mínimo de inactividad por mantenimiento o actualización.

Soporte:

El servicio de soporte técnico está disponible de lunes a viernes, en un horario de 8:00 am a 5:00 pm. Las solicitudes de soporte deben ser enviadas a través del correo de contacto habilitado, donde los usuarios podrán detallar sus problemas o consultas.

Responsabilidades:

Es responsabilidad de los usuarios proporcionar información clara y detallada al equipo de soporte cuando surjan problemas, para facilitar una resolución rápida. El equipo se encargará de mantener la comunicación durante todo el proceso.